

Tipus recursius de dades (II)

Departament de Ciències de la Computació

Universitat Politècnica de Catalunya



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Politècnica Superior d'Enginyeria
de Vilanova i la Geltrú

Índex

1 Tipus recursius de dades (II)

- Implementació de llistes
- Implementació dels arbres binaris

Definició de la classe `Llista` (I)

- La implementació amb nodes enllaçats que veurem és una mica diferent de `list` (STL de C++).
- Implementarem **l·listes amb punt d'interès**, en lloc d'iterador. No veurem com s'implementen els iteradors.
- Funcionalitats similars, algunes restriccions.

Llistes amb punt d'interès

Podem:

- Desplaçar endavant i enrere el punt d'interès
 - Afegir i eliminar just al punt d'interès
 - Consultar i modificar l'element al punt d'interès
-
- Implementació: atribut (privat) de tipus punter a node
 - Modularitat: el punt d'interès forma part del tipus, no és un tipus a part
 - Efecte lateral: queda modificat si es modifica en una funció que rep la llista per referència no `const`

Definició de la classe `Llista` (II)

```
template <class T> class Llista {  
private:  
    struct node_llista {  
        T info;  
        node_llista* seg;  
        node_llista* ant;  
    };  
    int longitud;  
    node_llista* primer_node;  
    node_llista* ultim_node;  
    node_llista* act;          // punter a punt d'interès  
    ...  
    // especificació i implementació d'ops. privades  
public:  
    // especificació i implementació d'ops. públiques  
};
```

Definició de la classe `Llista` (III)

- Punters per accés ràpid al primer i últim, i al punt d'interès.
- Si `act==NULL`, vol dir “punt d'interès sobre l'element fictici posterior a l'últim”.
- Si `act!=NULL`, apunta a l'element consultable de la llista (actual).

Definició de la classe `Llista` (III)

- Punters per accés ràpid al primer i últim, i al punt d'interès.
- Si `act==NULL`, vol dir “punt d'interès sobre l'element fictici posterior a l'últim”.
- Si `act!=NULL`, apunta a l'element consultable de la llista (actual).
- Per conveni, llista buida: `longitud` igual a 0, i els tres punters (`primer_node`, `ultim_node` i `act`) a `NUL`L.
- Llista amb un element: `longitud` 1 i únic altre cas en què `primer_node == ultim_node`.

Definició de la classe `Llista` (III)

- Punters per accés ràpid al primer i últim, i al punt d'interès.
- Si `act==NULL`, vol dir “punt d'interès sobre l'element fictici posterior a l'últim”.
- Si `act!=NULL`, apunta a l'element consultable de la llista (actual).
- Per conveni, llista buida: `longitud` igual a 0, i els tres punters (`primer_node`, `ultim_node` i `act`) a `NUL`L.
- Llista amb un element: `longitud` 1 i únic altre cas en què `primer_node == ultim_node`.
- “cap a la dreta” $\equiv$ cap a l'últim; “cap a l'esquerra” $\equiv$ cap al primer; “a la dreta de tot” $\equiv$ sobre l'element fictici del final.

Mètodes públics: constructors (I)

```
Llista()  
/* Pre: cert */  
/* Post: El resultat és una llista buida */  
{  
    longitud = 0;  
    primer_node = NULL;  
    ultim_node = NULL;  
    act = NULL;  
}
```

Mètodes públics: constructors (II)

```
Llista(const Llista& original)
/* Pre: cert */
/* Post: El resultat és una còpia d'original */
{
    longitud = original.longitud;
    primer_node = copia_node_llista(original.primer_node,
                                    original.act, ultim_node, act);
}
```

Mètodes públics: destructor

```
~Llista()  
// Destructeur: Esborra automàticament els  
// objectes locals en sortir d'un àmbit de  
// visibilitat  
{  
    esborra_node_llista(primer_node);  
}
```

Mètodes públics: Redefinició de l'assignació

```
Llista& operator=(const Llista& l)
/* Pre: cert */
/* Post: El p.i. passa a ser una còpia de l
        i el contingut anterior del p.i. ha estat
        esborrat (excepte si el p.i. i l ja eren
        el mateix objecte) */
{
    if (this != &l) {
        longitud = l.longitud;
        esborra_node_llista(primer_node);
        primer_node = copia_node_llista(l.primer_node,
                                         l.act, ultim_node, act);
    }
    return *this;
}
```

Mètodes públics: modificadors (I)

```
void l_buida ()  
/* Pre: cert */  
/* Post: El p.i. passa a ser una llista sense  
elements i qualsevol contingut anterior del  
p.i. ha estat esborrat */  
{  
    esborra_node_llista(primer_node);  
    longitud = 0;  
    primer_node = NULL;  
    ultim_node = NULL;  
    act = NULL;  
}
```

Mètodes públics: modificadors (II)

```
void afegir(const T& x);  
/* Pre: cert */  
/* Post: El p.i. és com el seu valor original  
però amb x afegit a l'esquerra del punt  
d'interès */
```

Mètodes públics: modificadors (II) cont.

```
void afegir (const T& x) {  
    node_llista* aux;  
    aux = new node_llista; // reserva espai nou element  
    aux->info = x;  
    aux->seg = act;  
    if (longitud == 0) {    // la llista és buida  
        aux->ant = NULL;  
        primer_node = aux;  
        ultim_node = aux;  
    }  
    else if (act == NULL) {  
        aux->ant = ultim_node;  
        ultim_node->seg = aux;  
        ultim_node = aux;  
    } ...  
}
```

Mètodes públics: modificadors (II) cont.

```
...  
else if (act == primer_node) {  
    aux->ant = NULL;  
    act->ant = aux;  
    primer_node = aux;  
}  
else {  
    aux->ant = act->ant;  
    (act->ant)->seg = aux;  
    act->ant = aux;  
}  
++longitud;  
}
```


Mètodes públics: modificadors (III)

```
void eliminar ()  
/* Pre: El p.i. és una llista no buida i el seu  
punt d'interès no està a la dreta de tot */  
/* Post: El p.i. és com el p.i. original però  
sense l'element on estava el punt d'interès i  
amb el nou punt d'interès apuntant al successor  
de l'element esborrat */
```

Mètodes públics: modificadors (III) cont.

```
void eliminar () {  
    node_llista* aux;  
    aux = act; // conserva accés al node actual  
    if (longitud == 1) {  
        primer_node = NULL;  
        ultim_node = NULL;  
    }  
    else if (act == primer_node) {  
        primer_node = act->seg;  
        primer_node->ant = NULL;  
    }  
    else if (act == ultim_node) {  
        ultim_node = act->ant;  
        ultim_node->seg = NULL;  
    } ...  
}
```

Mètodes públics: modificadors (III) cont.

```
...  
else {  
    (act->ant)->seg = act->seg;  
    (act->seg)->ant = act->ant;  
}  
act = act->seg; // avança el punt d'interès  
delete aux; // allibera l'espai de l'element esborrat  
—longitud;  
}
```

Mètodes públics: modificadors (IV)

```
void concat (Llista& l)  
/* Pre: l = L */  
/* Post: El p.i. té els seus elements originals  
seguits pels de L, l queda buida, i el punt  
d'interès del p.i. passa a ser el primer element */
```

Concatenació més eficient que la basada en `afegir`

Mètodes públics: modificadors (IV) cont.

```
void concat (Llista& l) {  
    if (l.longitud > 0) { // l buida --> no cal fer res  
        if (longitud == 0) {  
            primer_node = l.primer_node;  
        }  
        else {  
            ultim_node->seg = l.primer_node;  
            (l.primer_node)->ant = ultim_node;  
        }  
        ultim_node = l.ultim_node;  
        longitud += l.longitud;  
        ...  
    }
```

Mètodes públics: modificadors (IV) cont.

```
...  
l.primer_node = NULL;  
l.ultim_node = NULL;  
l.act = NULL;  
l.longitud = 0;  
}  
act = primer_node;  
}
```

Mètodes públics: consultors (I)

```
bool es_buida() const
/* Pre: cert */
/* Post: El resultat indica si el p.i. és una
    llista buida o no */
{
    return primer_node == NULL;
}

int mida() const
/* Pre: cert */
/* Post: El resultat és el nombre d'elements de
    la llista p.i. */
{
    return longitud;
}
```

Noves ops. per consultar/modificar el punt d'interès (I)

```
T actual() const    // equival a consultar *it
/* Pre: El p.i. és una llista no buida i el
   seu punt d'interès no està sobre l'element fictici
   del final */
/* Post: El resultat és l'element apuntat pel punt
   d'interès del p.i. */
{
    return act->info;
}
```


Noves ops. per consultar/modificar el punt d'interès (II)

```
void modifica_actual(const T &x)    // equival a fer *it=x
/* Pre: El p.i. és una llista no buida i el seu
   punt d'interès no està a la dreta de tot */
/* Post: El p.i. és com el seu valor original,
   però amb x reemplaçant l'element actual */
{
    act->info = x;
}
```

Noves ops. per moure el punt d'interès (I)

```
void inici()    // equival a fer it=l.begin()
/* Pre: cert */
/* Post: El punt d'interès del p.i. està situat
a sobre del primer element de la llista , o a la
dreta de tot si la llista és buida */
{
    act = primer_node;
}
```

Noves ops. per moure el punt d'interès (II)

```
void fi()    // equival a fer it=l.end()  
/* Pre: cert */  
/* Post: El punt d'interès del p.i. està situat  
        a la dreta de tot (sobre elem. fictici del final) */  
{  
    act = NULL;  
}
```

Noves ops. per moure el punt d'interès (III)

```
void avança()    // equival a fer ++it
/* Pre: El punt d'interès del p.i. no està a
   la dreta de tot */
/* Post: El punt d'interès del p.i. apunta al successor
   de l'element al qual apuntava originalment, és a dir,
   es mou cap a la dreta del seu valor original */
{
    act = act->seg;
}
```

Noves ops. per moure el punt d'interès (IV)

```
void retrocedeix()    // equival a fer —it
/* Pre: El punt d'interès del p.i. no és el
   primer element de la llista */
/* Post: El punt d'interès del p.i. està situat
   una posició més a l'esquerra que al valor
   original del p.i. */
{
    if (act == NULL) act = ultim_node;
    else act = act->ant;
}
```

Noves ops. per moure el punt d'interès (V)

```

bool dreta_de_tot() const // equival a comparar it==l.end()
/* Pre: cert */
/* Post: El resultat indica si el punt d'interès
    del p.i. està a la dreta de tot */
{
    return act == NULL;
}

bool sobre_el_primer() const // equival a comparar it==l.begin()
/* Pre: cert */
/* Post: El resultat indica si el punt d'interès
    del p.i. està a sobre del primer element del p.i.
    o està a la dreta de tot si la llista és buida */
{
    return act == primer_node;
}

```

Mètodes privats: per copiar cadenes de nodes (I)

```
static node_llista* copia_node_llista(node_llista* m,  
    node_llista* oact, node_llista* &u, node_llista* &a)  
/* Pre: cert */  
/* Post: Si m és NULL, el resultat, u i a són NULL;  
    en cas contrari, el resultat apunta al primer node  
    d'una cadena de nodes que són còpia de la cadena  
    que té el node apuntat per m com a primer, u apunta  
    a l'últim node, a és o bé NULL si oact no apunta a  
    cap node de la cadena que comença amb m o bé apunta  
    al node còpia del node apuntat per oact */
```

Mètodes privats: per copiar cadenes de nodes (I) cont.

```

static node_llista* copia_node_llista(node_llista* m,
    node_llista* oact, node_llista* &u, node_llista* &a)
{
    node_llista* n;
    if (m == NULL) {n=NULL; u=NULL; a=NULL;}
    else {
        n = new node_llista;
        n->info = m->info;
        n->ant = NULL;
        n->seg = copia_node_llista(m->seg, oact, u, a);
        if (n->seg == NULL) u = n;
        else (n->seg)->ant = n;
        if (m == oact) a = n;
    }
    return n;
}

```


Mètodes privats: per esborrar cadenes de nodes

```
static void esborra_node_llista (node_llista* m)
/* Pre: cert */
/* Post: No fa res si m és NULL, sinó allibera
    espai dels nodes de la cadena que té el node
    apuntat per m com a primer */
{
    if (m != NULL) {
        esborra_node_llista (m->seg);
        delete m;
    }
}
```

Exercici: Feu la versió iterativa d'aquesta funció.

Definició de la classe `Arbre`

- La implementació amb nodes enllaçats que veurem és molt semblant a la classe `arbreBin`.

Definició de la classe `Arbre`

- La implementació amb nodes enllaçats que veurem és molt semblant a la classe `arbreBin`.
- Cada node (`struct`) conté dos punters a node.
- Un arbre buit tindrà l'atribut `arrel` nul.

Implementació de la classe `Arbre` (I)

```
template <class T> class Arbre {  
private:  
    struct node_arbre {  
        T info;  
        node_arbre* segE;  
        node_arbre* segD;  
    };  
    node_arbre* arrel;  
  
    ...    // especificació d'operacions privades  
  
public:  
  
    ...    // especificació d'operacions públiques  
};
```

Mètodes públics: constructors i destructor (I)

```
Arbre()  
/* Pre: cert */  
/* Post: El resultat és un arbre sense cap element */  
{  
    arrel = NULL;  
}  
  
Arbre(const Arbre &original)  
/* Pre: cert */  
/* Post: El resultat és un arbre còpia d'original */  
{  
    arrel = copia_node_arbre(original.arrel);  
}
```

Mètodes públics: constructors i destructor (II)

```

Arbre(const T &x, const Arbre &ae, const Arbre &ad)
/* Pre: cert */
/* Post: El resultat és un arbre amb l'element x com a
        arrel, ae com a fill esquerre i a2 com a fill dret */
{
    arrel = new node_arbre;
    arrel->info = x;
    arrel->segE = copia_node_arbre(ae.arrel);
    arrel->segD = copia_node_arbre(ad.arrel);
}

~Arbre()
// Destructor: esborra automàticament els objectes
// locals en sortir d'un àmbit de visibilitat
{
    esborra_node_arbre(arrel);
}

```

Mètodes públics: consultors (I)

```
T arrel() const
/* Pre: El paràmetre implícit no és buit */
/* Post: El resultat és l'arrel del paràmetre implícit */
{
    return arrel->info;
}

bool es_buit() const
/* Pre: cert */
/* Post: El resultat indica si el p.i. és buit o no */
{
    return (arrel == NULL);
}
```

Mètodes públics: consultors (I)

```
Arbre fe() const
/* Pre: El paràmetre implícit no és buit */
/* Post: El resultat és el fill esquerre del p.i. original */
{
    Arbre a;
    a.arrel = copia_node_arbre(arrel->segE);
    return a;
}
```

```
Arbre fd() const
/* Pre: El paràmetre implícit no és buit */
/* Post: El resultat és el fill dret del p.i. original */
{
    Arbre a;
    a.arrel = copia_node_arbre(arrel->segD);
    return a;
}
```


Mètodes públics: operador d'assignació i modificador

```

Arbre& operator=(const Arbre &original)
/* Pre: cert */
/* Post: Operació d'assignació d'arbres */
{
    if (this != &original) {
        node_arbre* aux;
        aux = copia_node_arbre(original.arrel);
        esborra_node_arbre(arrel);
        arrel = aux;
    }
    return *this;
}

void a_buit()
/* Pre: cert */
/* Post: El paràmetre implícit no té cap element */
{
    esborra_node_arbre(arrel);
    arrel = NULL;
}

```

Mètodes privats: per copiar jerarquies de nodes

```
static node_arbre* copia_node_arbre(node_arbre* m)
/* Pre: cert */
/* Post: Si m és NULL, el resultat és NULL;
   en cas contrari, el resultat apunta al node arrel
   d'una jerarquia de nodes que és una còpia de la
   jerarquia de nodes que té el node apuntat per m
   com a arrel */
{
    node_arbre* n;
    if (m == NULL) n = NULL;
    else {
        n = new node_arbre;
        n->info = m->info;
        n->segE = copia_node_arbre(m->segE);
        n->segD = copia_node_arbre(m->segD);
    }
    return n;
}
```

Notem l'operador = del tipus T usat com a una operació de còpia.

Mètodes privats: per esborrar jerarquies de nodes

```
static void esborra_node_arbre(node_arbre* m)
/* Pre: cert */
/* Post: No fa res si m és NULL; en cas contrari,
    allibera espai de tots els nodes de la jerarquia
    que té el node apuntat per m com a arrel */
{
    if (m != NULL) {
        esborra_node_arbre(m->segE);
        esborra_node_arbre(m->segD);
        delete m;
    }
}
```